ケモインフォマティクス部会
ハンズオン講習会

基礎から学ぶ
分子構造生成モデルの仕組み

RNN・LSTM による SMILES 生成

Kai OSHIRO
WPI-ICReDD, Hokkaido University
November 9, 2026

目次

1. 系列データと SMILES　p.4–11

- 文字列・語彙・トークン
- 整数 index・one-hot ベクトル・パディング
- 学習データの準備

2. RNN の構造と学習　p.12–30

- 隠れ状態と出力層
- 教師強制・確率・損失関数
- RNN の課題

3. LSTM の記憶セルとゲート　p.31–41

- 記憶セル・3 つのゲート・記憶セルの候補
- 記憶セルの保持・更新・隠れ状態の計算
- 過去の情報を伝える方法

4. SMILES の生成　p.42–67

- 系列モデルを用いた分子生成
- 4 時刻にわたる生成過程
- 全体のまとめ（p.67）・参考資料（p.68）

文字列を入力に変え、次の文字の予測を学習し、その予測から分子を生成する流れを追う。

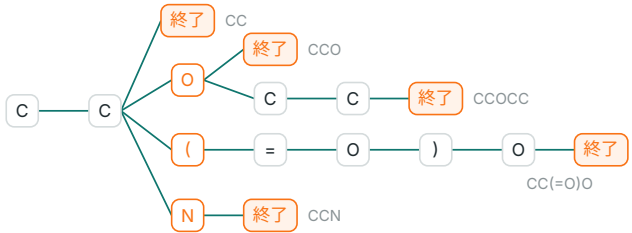
例題のデータセット

ここでは、SMILES の表記がいずれも "CC" から始まる 5 分子を例に、系列モデルの仕組みについて学ぶ。

データセット $\mathcal{D}$

No.	分子	SMILES
1	エタン	CC
2	エタノール	CCO
3	ジエチルエーテル	CCOCC
4	酢酸	CC(=O)O
5	エチルアミン	CCN

"CC" から先の分岐



5 分子の SMILES はすべて "CC" から始まるが、続く文字は分子によって異なる。"CC" の次にくる文字の確率を例に、系列モデルの学習や分子生成の流れについて学ぶ。

次の文字	終了	O	(	N	合計
分子数	1	2	1	1	5
割合	20%	40%	20%	20%	100%

実際には SMILES の文字以外に開始や終了を表す特殊記号も加える（p.7）。

Part 1

系列データと SMILES

分子を文字列として表し、次に続く文字を予測する

SMILES による分子の表現

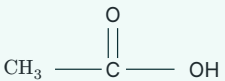


SMILES（ Simplified Molecular Input Line Entry System）は分子構造を 1 行の文字列で表す記法。

- 原　子：C は炭素、O は酸素、N は窒素。
- 結　合：単結合は通常省略する。“=” は二重結合、“#” は三重結合を表す。
- 分　岐：枝分かれする部分を “(“ と “)” で囲む。
- 水　素：通常は省略し、原子価から補う。
- 　環　：同じ数字を付けた 2 原子を結ぶ。
- 芳香族：炭素は小文字の c で表す。
- 角括弧：水素数や電荷を明示できる。例えば [NH4+] はアンモニウムイオン。

酢酸 “CC(=O)O”

C	C	(	=	O	)	O



2番目の "C" から "O" が分岐し、二重結合でつながる。

環と芳香族の例：ベンゼン



c1ccccc1

小文字 c：芳香族の炭素

今回の 5 分子には含まれない。

系列データの定義



文章や SMILES のように、順序に意味があるデータの並びを**系列データ**と呼びます。

$$x = (x_1, x_2, ..., x_T) \in \mathcal{X}^T$$

- T：系列の長さ（系列長）
- $\mathcal{X}$：各要素 x_t が取りうる値の集合

$\in$ は「集合に含まれる」の意味。$\mathcal{X}^T$ は $\mathcal{X}$ の要素を T 個並べた系列すべての集合。

系列全体を x とし、それを区切った各要素を先頭から $x_1, x_2, ..., x_T$ と表す。各要素 x_t の t を時刻と呼ぶ（入力の順番に対応）。

文字列の例

エタノール：$x = \mathrm{CCO},\quad T = 3$

酢酸：$x = \mathrm{CC(=O)O},\quad T = 7$

文字列に現れる語彙

語彙は扱う文字・記号の集合、今回は 6 種類。

$\mathcal{X} = \{(,), =, \mathrm{C}, \mathrm{N}, \mathrm{O}\}$

開始・終了・パディングの特殊記号



トークンの定義

文字列を分割した後の各要素を**トークン**と呼ぶ。
例えば、SMILES 文字列に含まれる Cl や Br など、
二文字で 1 トークンとする場合もある。

$\langle \mathrm{sos} \rangle$ = “!” ：文字列の開始を示す記号。

$\langle \mathrm{eos} \rangle$ = “?” ：文字列の終了を示す記号。

$\langle \mathrm{pad} \rangle$ = “␣” ：系列長を揃えるための記号。

語彙サイズ（$V = 9$）

6 種類の通常トークンと 3 種類の特殊記号で、
このデータセットの語彙サイズ V は 9 となる。

$$\mathcal{X} = \{(,), =, \mathrm{C}, \mathrm{N}, \mathrm{O}\}$$

$$\tilde{\mathcal{X}} := \{!, ?, \sqcup\} \cup \mathcal{X}$$

各トークンに 0〜8 の index を割り振る。index は計算に用いる識別番号であり、数値の大小に意味はない（p.9）。

重複を除いたトークンの集合を語彙と呼び、ここでは SMILES 由来のトークンに特殊記号のものを加える。

開始・終了記号を加えた学習用の系列



モデルに生成の開始と終了を伝えるため、各 SMILES の先頭に “!”、末尾に “?” を付ける。これらは学習用に加える記号で、SMILES 本体には含めない。

分子	SMILES	開始・終了記号を加えた系列
エタン	CC	!CC?
エタノール	CCO	!CCO?
ジエチルエーテル	CCOCC	!CCOCC?
酢酸	CC(=O)O	!CC(=O)O?
エチルアミン	CCN	!CCN?

語彙は重複を除いたトークンの集合。特殊記号を加えた語彙が $\tilde{\mathcal{X}}$ で、語彙サイズは $V = 9$ となる。

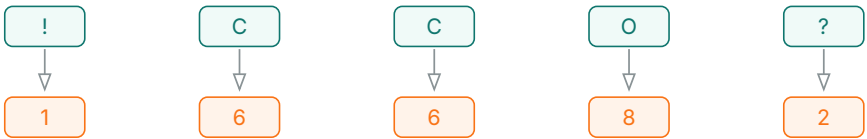
$$\tilde{\mathcal{X}} = \{\sqcup, !, ?, (,), =, \mathrm{C}, \mathrm{N}, \mathrm{O}\}, \quad V = 9$$

語彙テーブルによるトークンの整数化



各トークンを数値に変換するため、それぞれに index を割り振る。この対応表を語彙テーブルと呼ぶ。

トークン	␣	!	?	(	)	=	C	N	O
index	0	1	2	3	4	5	6	7	8

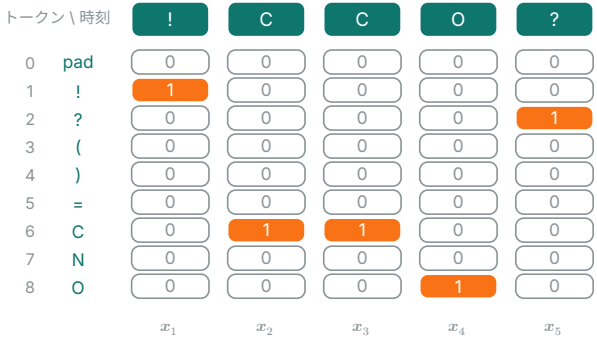


語彙サイズは $V = 6 + 3 = 9$。index はトークンを識別する番号であり、その大小に意味はない。数値の大小をトークンどうしの関係として扱わないよう、index を one-hot ベクトルに変換する。

one-hot ベクトルによるトークンの表現



one-hot ベクトルとは、語彙サイズと等しい次元をもったベクトルであり、
index に対応する位置の成分だけが 1 で、他は 0 のベクトル。



1 列につき 1 つの one-hot ベクトル

$\mathbf{1}_d$：index d の成分だけ 1、他は 0

$\mathbf{1}_d \in \{0,1\}^9$, $\sum_{k=0}^{8} (\mathbf{1}_d)_k = 1$

“C” は $\mathbf{1}_6$、“O”（酸素）は $\mathbf{1}_8$

2 列目と 3 列目は同じ “C”

図は開始・終了を含む系列全体の one-hot 表現。
それぞれの列が 1 時刻ごとの入力に対応する。

パディングによる系列長の統一

複数の入力データをまとめたものをミニバッチと呼ぶ。ニューラルネットワークで効率よく計算するために用いられる。系列データでは短い系列の末尾に $\langle \mathrm{pad} \rangle$ を加え、ミニバッチ内の最長の系列と同じ長さにする。

	$t=1$	$t=2$	$t=3$	$t=4$	$t=5$	$t=6$	$t=7$	$t=8$	$t=9$
エタン	!	C	C	?	␣	␣	␣	␣	␣
エタノール	!	C	C	O	?	␣	␣	␣	␣
ジエチルエーテル	!	C	C	O	C	C	?	␣	␣
酢酸	!	C	C	(	=	O	)	O	?
エチルアミン	!	C	C	N	?	␣	␣	␣	␣

開始・終了を含む整数列：5（データ数）$\times$ 9（最大系列長）

one-hot ベクトル：5（データ数）$\times$ 9（語彙数）$\times$ 9（最大系列長）

文字列を数値の配列に変換できた。次はこの配列を処理する RNN の仕組みと学習方法を学ぶ。

Part 2

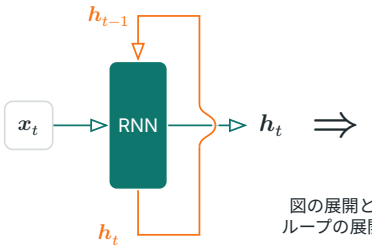
RNN の構造と学習

過去の入力を隠れ状態に反映し、次に続くトークンを予測する

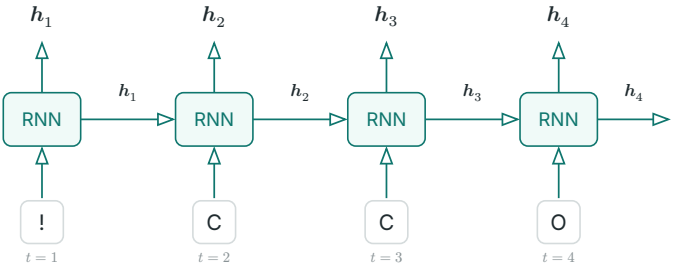
RNN の構造

系列モデルはデータの順序を考慮して予測するモデル。RNN（Recurrent Neural Network）はその一つで、隠れ状態 h_t を通して、過去の時刻の情報を次の時刻に伝える仕組みを持っている。

展開前のループ構造



時間方向への展開



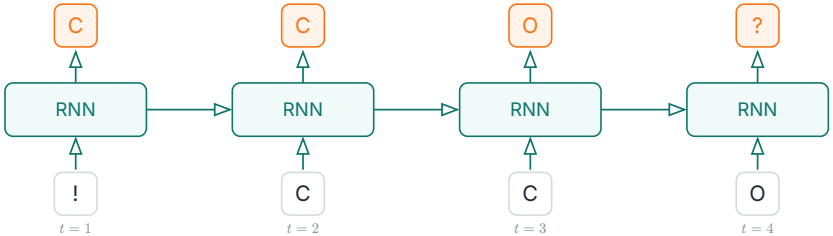
図の展開と
ループの展開

h_t は次の時刻と出力（予測）側の両方へ渡される
固定長のベクトルである。

全時刻で重みを共有するため、系列が長くなっても
パラメータ数は増えない。

RNN の学習と教師強制

系列データを扱う際の課題の一つは、次に続く値やトークンを予測することである。ここでは、それまでに入力された文字列をもとに、各時刻で次のトークンを予測するように RNN を学習させる。



入力と正解

入力：$(!, C, C, O)$ / 正解：$(C, C, O, ?)$

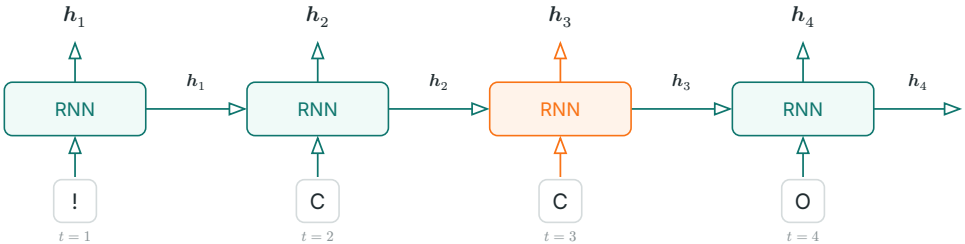
正解は、入力を 1 トークン先へずらして作る。

学習中は直前の予測に関係なく、常に正しいトークンを入力する。この学習方法を教師強制（teacher forcing）と呼ぶ。

生成時にはモデルが直前の時刻に選んだトークンを次の入力とし、新しい系列を生成できる。

$t = 3$ における次のトークンの予測

エタノール “CCO” を例に、$t = 3$ の場合の計算を考える。この時刻で予測したい次のトークンは “O” である。



時刻を表す添字

$t = 1$ では開始記号 "!"、$t = 2$ では最初の "C"、$t = 3$ では 2 個目の "C" を入力する。開始記号も 1 時刻として数える。

RNN レイヤの入力と隠れ状態

$\boldsymbol{x}_3$：$t = 3$ での入力 “C”
$\boldsymbol{h}_2$：1 時刻前（$t = 2$）の隠れ状態
$\boldsymbol{h}_3$：$t = 3$ で更新した隠れ状態

入力ベクトル・隠れ状態・初期状態

入力ベクトルの次元は語彙のサイズ $V = 9$ に等しく、隠れ状態の次元 H は設計時に決めるハイパーパラメーターである。ここでは $H = 3$ とし、入力ベクトル $\boldsymbol{x}_3$ と隠れ状態 $\boldsymbol{h}_2$ を示す。太字の記号はベクトルを表す。



9 成分の入力ベクトル

$$\boldsymbol{x}_3 = (0, 0, 0, 0, 0, 0, 1, 0, 0)^T$$

index 6 が “C” に対応し、$\boldsymbol{x}_3$ はその成分だけが 1 になる。T は転置を表し、ここでは列ベクトルを示す。

3 成分の隠れ状態

$$\boldsymbol{h}_2 = \begin{pmatrix} h_{2,1} \\ h_{2,2} \\ h_{2,3} \end{pmatrix}$$

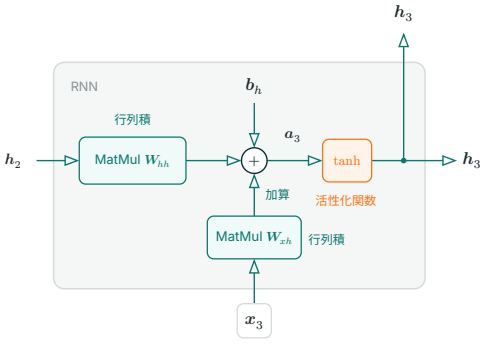
$h_{2,1}$：$t = 2$ のときの 1 番目の成分。

初期状態 $\boldsymbol{h}_0$

独立した系列を入力に用いる場合、通常は $\boldsymbol{h}_0 = \boldsymbol{0}$ とする。すなわち、$\boldsymbol{h}_0 = (0, 0, 0)^T$。

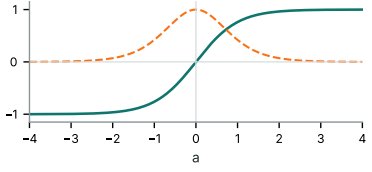
RNN の更新式と行列の形状

隠れ状態と入力のベクトルはそれぞれ次元がことなるため、別々の重みパラメーターを用いてこれらの次元を揃える。その後、活性化関数である tanh を適用して、隠れ状態を更新する。



$$\underbrace{\boldsymbol{a}_3}_{H\times 1} = \underbrace{\boldsymbol{W}_{xh}}_{H\times V}\underbrace{\boldsymbol{x}_3}_{V\times 1} + \underbrace{\boldsymbol{W}_{hh}}_{H\times H}\underbrace{\boldsymbol{h}_2}_{H\times 1} + \underbrace{\boldsymbol{b}_h}_{H\times 1}$$

$$\underbrace{\boldsymbol{h}_3}_{H\times 1} = \underbrace{\tanh(\boldsymbol{a}_3)}_{H\times 1}$$

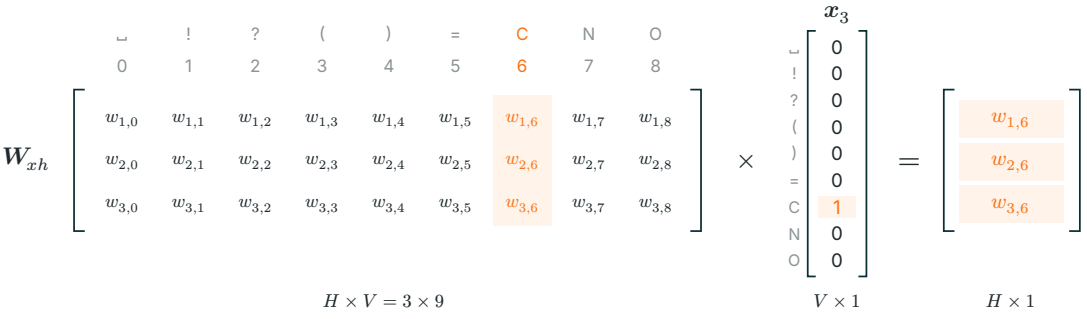


実線：tanh。破線：微分（曲線の傾き）。

tanh は各成分を −1 より大きく 1 より小さい値に変換する。$\boldsymbol{a}_3$ は加算結果、$\boldsymbol{b}_h$ はバイアス。重みとバイアスを全時刻で共有し、学習で更新する。

重みパラメーターの役割

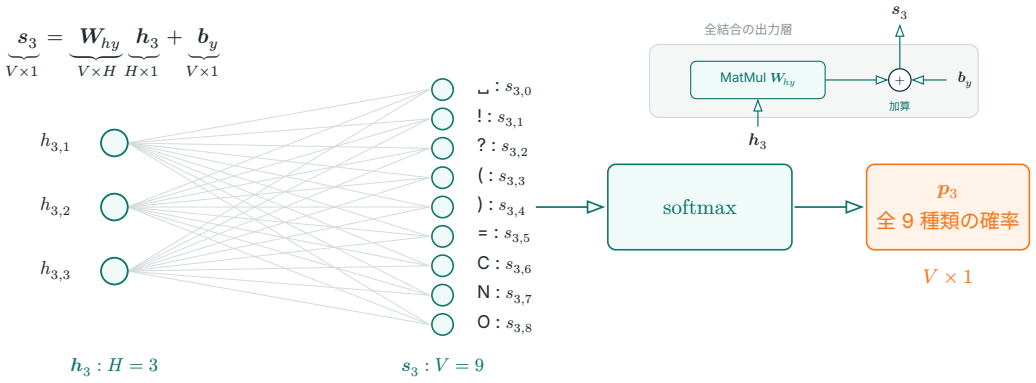
one-hot ベクトルの場合、$\boldsymbol{W}_{xh}$ 行列の対応する列（入力 "C" の index と同じ位置）を選択するような操作に対応する。



行列と入力ベクトルの積 $\boldsymbol{W}_{xh}\boldsymbol{x}_3$ は、$H \times 1$ の列ベクトルとなる。行は隠れ状態の成分、列は語彙の index に対応する。

出力層によるスコアの計算

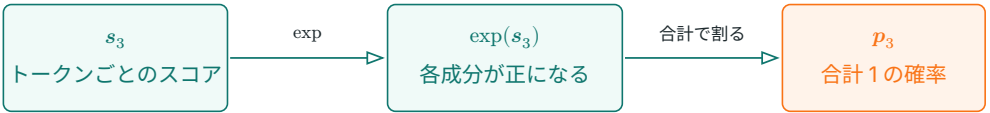
更新した隠れ状態をもとに次のトークンを予測するため、RNN レイヤの後に**全結合の出力層**を配置する。隠れ状態の H 成分から、各トークンに対応する V 個のスコアを計算する。



ここでは、隠れ状態の次元から、語彙のサイズと同じ次元のベクトルへの変換を行う。その後、softmax 関数を適用すれば、各トークンに対応する値を 0 より大きく 1 より小さく、その合計が 1 となるように変換できる。

softmax によるスコアの確率への変換

出力層のスコアを、次のトークンを選ぶための確率に変換する。各スコアに指数関数を適用して正の値にし、それらの合計で割ると、各トークンの予測確率が得られる。



$$p_{3,k} = \frac{\exp(s_{3,k})}{\sum_{j=0}^{V-1} \exp(s_{3,j})}$$

k：求めたいトークンの index

j：全語彙の合計を取るための index

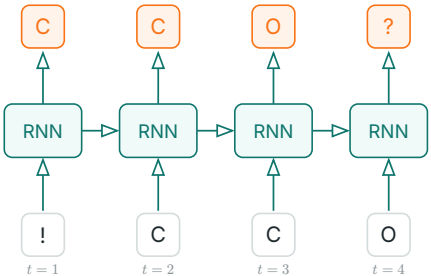
下付き 3：時刻 $t = 3$ での予測

これで入力したトークンの次にくるトークンの確率分布に対応する値が得られた。$t = 3$ の場合は "O" の予測確率が高くなるように学習を行う。

系列全体の予測確率

エタノール（"!CCO?"）を学習する場合について考える。開始記号 "!" を与えたときに正解のトークン全体の系列が出力される確率 P は、各時刻でそれぞれの正解トークンを予測する確率の積に対応する。

各時刻の正解トークンの予測



図は 5 分子のうちの 1 例。各分子で、実際に続くトークンを正解として学習する。

条件付き確率の積への展開

$$
\begin{aligned}
P &= p(\mathrm{C},\mathrm{C},\mathrm{O},?|!) \\
&= p(\mathrm{C},\mathrm{O},?|!,\mathrm{C})p(\mathrm{C}|!) \\
&= p(\mathrm{O},?|!,\mathrm{C},\mathrm{C})p(\mathrm{C}|!,\mathrm{C})p(\mathrm{C}|!) \\
&= p(?|!,\mathrm{C},\mathrm{C},\mathrm{O})p(\mathrm{O}|!,\mathrm{C},\mathrm{C})p(\mathrm{C}|!,\mathrm{C})p(\mathrm{C}|!)
\end{aligned}
$$

縦線の右は条件、左はその条件下で生成する文字列。

ここでは、確率の乗法定理 $p(A,B) = p(A|B)p(B)$ を用いた。

各時刻で正しく正解トークンを予測する学習を通して、"CCO?" などの系列全体が出現する確率をデータセットに近づける。

系列確率と負の対数損失

確率を多数掛けると、計算機で扱いにくいほど小さくなる。対数で積を和に変えると、小さな積を直接扱わずに計算できる。

系列全体の確率と対数

$$
\begin{aligned}
P &= p(\mathrm{C},\mathrm{C},\mathrm{O},?|!) \\
&= p(\mathrm{C}|!)p(\mathrm{C}|!\mathrm{C})p(\mathrm{O}|!,\mathrm{C},\mathrm{C})p(?|!,\mathrm{C},\mathrm{C},\mathrm{O}) \\
&= p_{1,6}p_{2,6}p_{3,8}p_{4,2}
\end{aligned}
$$

$$
\begin{aligned}
\log P &= \log p_{1,6} + \log p_{2,6} + \log p_{3,8} + \log p_{4,2} \\
&= \sum_{t=1}^{4} \log p_{t,y_t}
\end{aligned}
$$

各時刻の損失の和

$$
\begin{aligned}
-\log P &= -\log p_{1,6} - \log p_{2,6} \\
&\quad - \log p_{3,8} - \log p_{4,2} \\
&= \ell_1 + \ell_2 + \ell_3 + \ell_4
\end{aligned}
$$

$$
\begin{aligned}
\ell_1 &= -\log p_{1,6} \\
\ell_2 &= -\log p_{2,6} \\
\ell_3 &= -\log p_{3,8} \\
\ell_4 &= -\log p_{4,2}
\end{aligned}
$$

記号と添字の意味

- t：予測する時刻
- k：語彙の index
- y_t：正解トークンの index
- $p_{t,k}$：時刻 t に index k へ与えた予測確率
- $\ell_t = -\log p_{t,y_t}$：時刻 t の損失
- C：6、O：8、?：2
- $\log$：底 e の自然対数

対数は確率の大小関係を保つ。さらに符号を反転すると、P を最大化する学習が $-\log P$ を最小化する学習に変わる。

交差エントロピーと各時刻の損失

交差エントロピーは各トークンの $-\log p$ を正解の確率 q で重み付けして足した値。重みの合計が 1 なので、これは q に対する平均（期待値）となる。語彙数 V で割る必要はない。

$$\ell_t = -\sum_{k=0}^{V-1} q_{t,k} \log p_{t,k} = -\log p_{t,y_t}$$

- t：時刻
- k：語彙の index
- V：語彙数（9）
- y_t：正解の index
- $q_{t,k}$：one-hot の k 成分

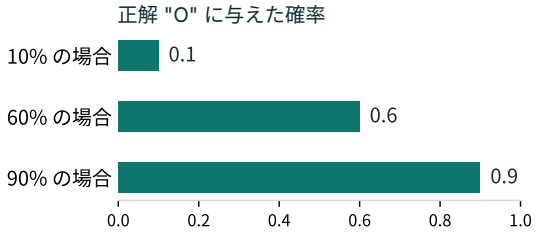
トークン / index	␣ / 0	! / 1	? / 2	(/ 3	) / 4	= / 5	C / 6	N / 7	O / 8
予測確率 $p_{3,k}$	0.02	0.01	0.08	0.03	0.02	0.04	0.10	0.10	0.60
正解 $q_{3,k}$	0	0	0	0	0	0	0	0	1

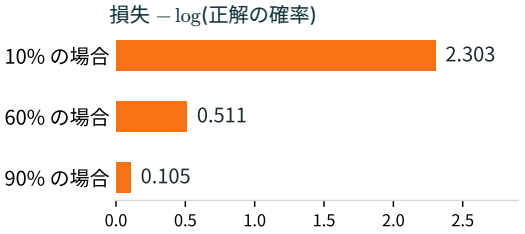
正解 “O” の予測確率だけが損失の計算に使われる

“!CC” の次が “O” なら $y_3 = 8$。正解の成分だけ $q_{3,8} = 1$、他は 0。説明用の予測値 $p_{3,8} = 0.60$ では $\ell_3 = -\log 0.60 \approx 0.51$。

正解の予測確率と損失の関係

交差エントロピーでは正解トークンにどれだけ高い確率を与えたかを評価する。
正解が "O" のとき、その予測確率と損失の対応を確かめる。





one-hot の正解に対する損失

$\ell_t = -\log p_{t,y_t}$ （log は自然対数）

y_t：正解トークンの index。

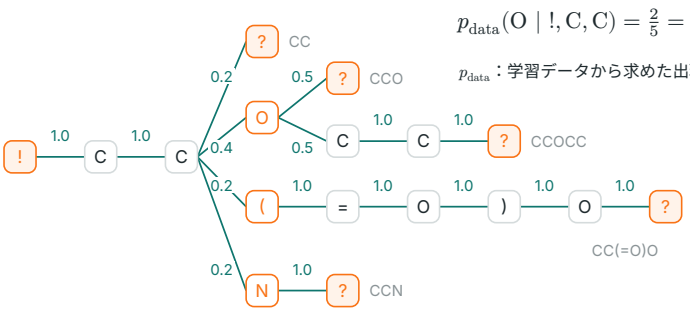
p_{t,y_t}：そのトークンに与えた確率。

正解に低い確率を与えた予測ほど、大きな損失になる。次は同じ入力履歴を持ち、正解が異なる学習例をまとめて、経験分布との関係を確認する。

学習データにおける条件付き確率

同じ "CC" を入力しても、次の正解トークンは分子によって異なる。学習の目標は、
"CC" の次に各トークンが現れる予測確率を、学習データ中の割合に近づけること。

モデルが "O" と予測する確率を学習データと同じ割合（0.4）に近づけたい。



現在のデータセット

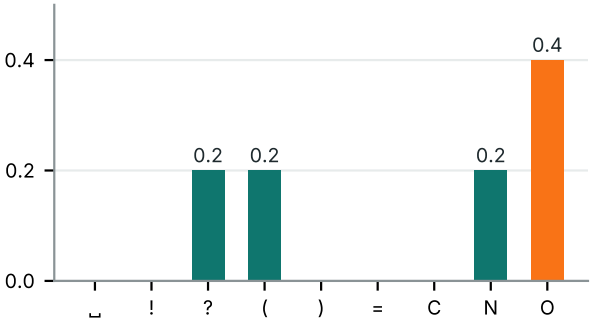
分子	SMILES
エタン	CC
エタノール	CCO
ジエチルエーテル	CCOCC
酢酸	CC(=O)O
エチルアミン	CCN

"O" が続くのはエタノールと
ジエチルエーテルの 2 分子。

"CCO" の次は "?" と "C" が 1 分子ずつなので、それぞれ 0.5 となる。学習時には各分子で実際に続く 1 トークンを正解とする。

予測分布と経験分布

同じ入力履歴を持つ学習例をまとめて考える。学習データから求めた各トークンの出現割合を確率とする分布を経験分布、モデルが各トークンに割り当てる確率の分布を予測分布と呼ぶ。



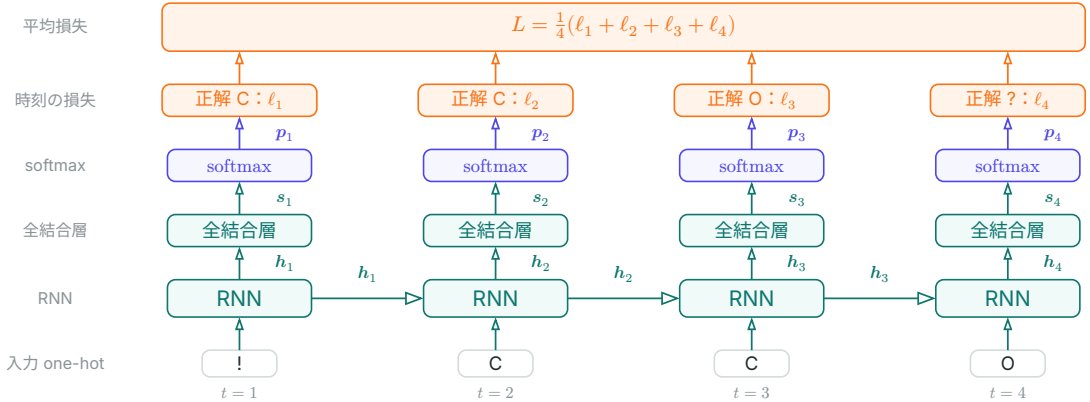
現在のデータセット

分子	SMILES
エタン	CC
エタノール	CCO
ジエチルエーテル	CCOCC
酢酸	CC(=O)O
エチルアミン	CCN

"CC" の次に "O" が現れたのは 5 分子中 2 分子なので、経験分布での "O" の確率は 0.4 となる。正解の one-hot ベクトルの平均がこの経験分布になる。各例の交差エントロピーの平均を最小にすることで、予測分布を経験分布に近づける。

各時刻の損失と平均損失 L

系列全体の負の対数確率は各時刻の損失の和で表せる。ここでは “CCO?” の 4 時刻分を平均し、学習で最小化する一つの値 L とする。

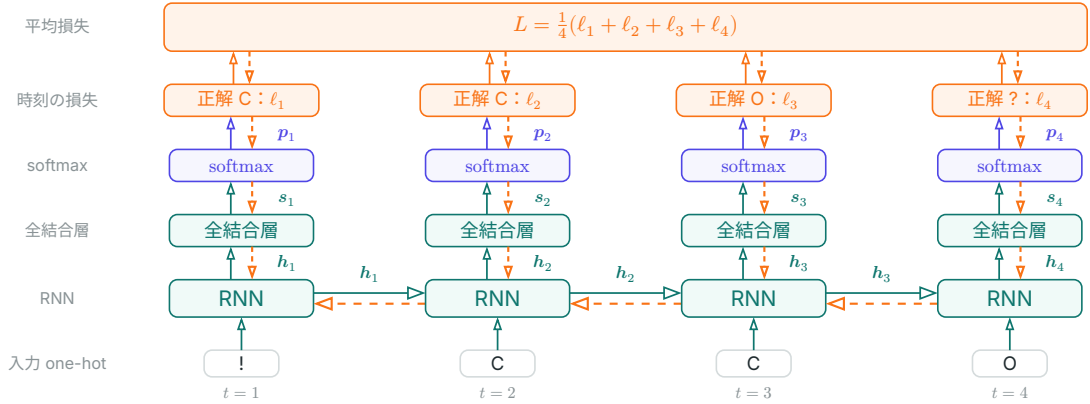


$$L = \frac{1}{4}\sum_{t=1}^{4} \ell_t = -\frac{1}{4}\log p(\mathrm{CCO?}|\mathrm{!})$$

全予測を一つの目的関数で評価する。平均にすると、予測位置の数による損失の大きさの違いを抑えられる。

平均損失 L の勾配と時間方向の逆伝播

θ：重み・バイアス全体。∇：勾配を表す記号。$\nabla_\theta L$：各パラメーターに対する L の変化率を並べたもの。

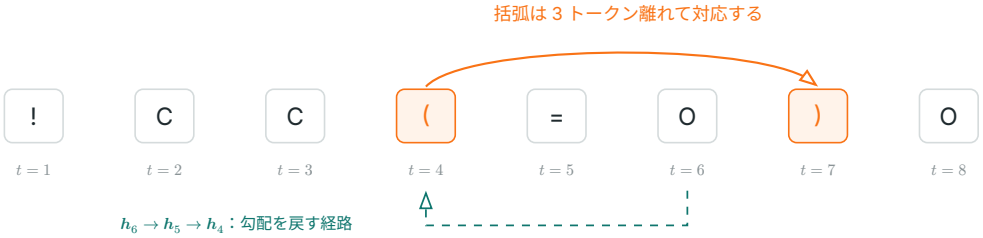


$$\nabla_\theta L = \frac{1}{4}\sum_{t=1}^{4} \nabla_\theta \ell_t = -\frac{1}{4}\nabla_\theta \log P$$

重みを固定して求めた各時刻の勾配を平均すると、L の勾配になる。オレンジの破線は逆伝播を表す。

離れたトークンの対応と勾配

酢酸 “CC(=O)O” では、開き括弧の情報を使って閉じ括弧を予測する。閉じ括弧の予測誤差から、前の入力を状態に残す処理も学習したい。



例題の 2 ステップ

時刻 4 に “(“ を入力し、時刻 6 に “)” を予測する。その誤差からの勾配は $h_6 \to h_5 \to h_4$ と 2 回の状態更新を戻る。

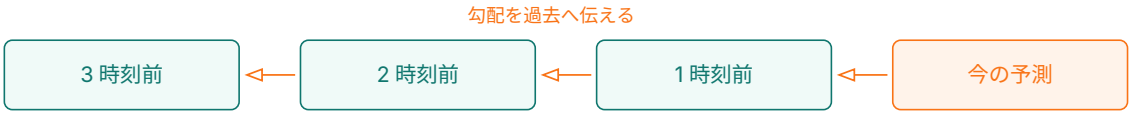
長い分岐や環の対応

実際の分子では、関連するトークンがもっと離れる。後の予測を改善するには、前の情報を保持する処理まで勾配が届く必要がある。

途中で勾配が極端に小さくなると、前の処理を改善する寄与が弱まり、離れた対応を学びにくくなる。

RNN の勾配消失と勾配爆発

後の時刻の予測誤差を減らすため、勾配を前の時刻へ伝える。伝わる途中で
各時刻の倍率が順に掛かり、遠い時刻ほど小さな縮小や拡大が積み重なる。



通常の NN では層数が、RNN では時刻間の距離が勾配を伝える計算の長さに関わる。
RNN は 1 層でも、離れたトークンの対応を学ぶには多くの時刻をさかのぼる。

勾配消失

倍率が 0.9 なら、50 回で $0.9^{50} \approx 0.005$。
遠い過去へ伝わる勾配が小さくなり、離れたトークンの
対応を学びにくくなる。

勾配爆発

倍率が 1.1 なら、50 回で $1.1^{50} \approx 117$。
大きな勾配で重みの更新が進みすぎ、
学習が不安定になる。

必要な時刻数はデータと課題による。系列や逆伝播を短く区切れるが、遠い依存関係を学ぶ寄与も失う。NN の層数を減らすことも万能な対策ではない。数値は倍率を一定とした例。

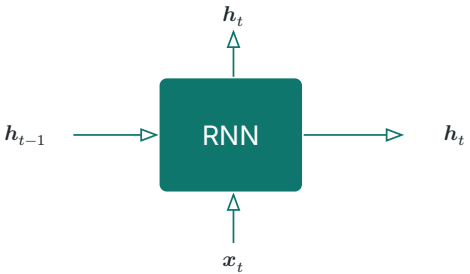
Part 3

LSTM の記憶セルとゲート

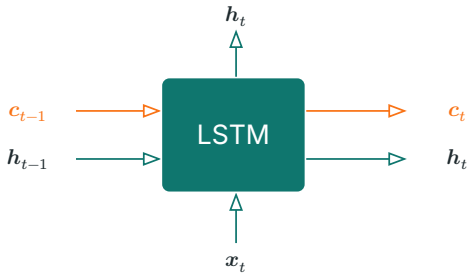
記憶セルに保持・追加する情報の量と、隠れ状態に出力する情報の量をゲートで調整する

RNN と LSTM の入出力

RNN の隠れ状態



LSTM の隠れ状態と記憶セル



共通の隠れ状態 h_t（$H \times 1$）

RNN も LSTM も隠れ状態を出力層と次時刻に渡す。
LSTM では記憶セルから予測に使う値を計算する。

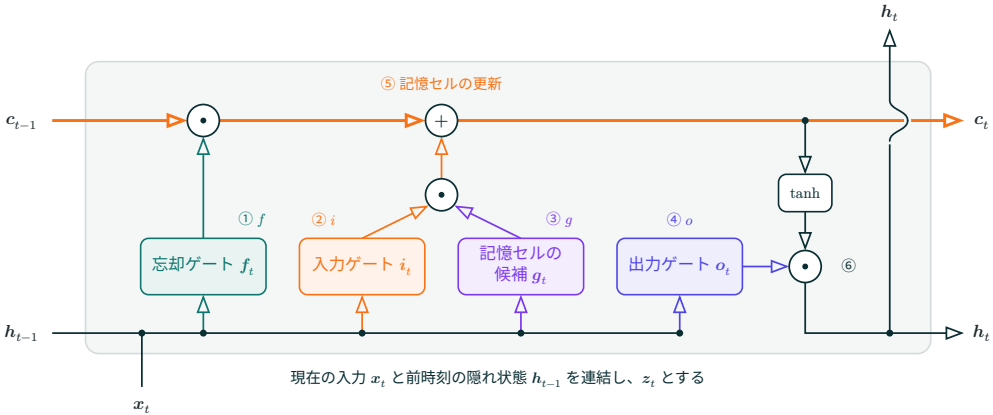
LSTM の記憶セル c_t

過去の情報を保持する $H \times 1$ のベクトル。ここでは $H = 3$。通常は $c_0 = \mathbf{0}$ とし、出力層へ直接渡さない。

LSTM：Long Short-Term Memory。出力層・softmax・損失は RNN と共通。

LSTM の全体構造

LSTM の内部を、記憶セルの更新と隠れ状態の計算に分けて見る。下の 4 つの計算のうち、①②④ は情報を通す割合を決めるゲート、③ は追加する情報の候補。

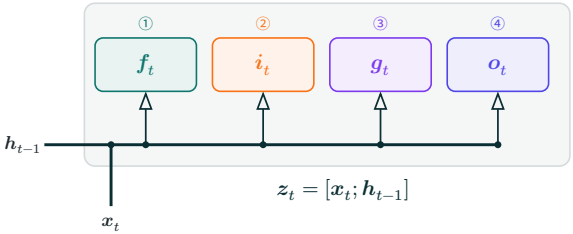


左から前時刻の c_{t-1}, h_{t-1}、下から今の入力 x_t。右へ新しい c_t, h_t、上へ予測に使う h_t。

入力と前時刻の隠れ状態の連結

図の下側ではゲートの値と記憶セルの候補を計算する。連結は 2 つのベクトルを縦に並べる操作。
$z_t = [x_t; h_{t-1}]$ と書く。入力と前の状態をまとめると、2 つの行列積を 1 つの式で表せる。

4 つの計算に共通する入力（p.33 の下側）



連結の説明用に $V = 9$・$H = 3$ とした例

$x_3 = (0, 0, 0, 0, 0, 0, 1, 0, 0)^\top$
C の one-hot：9 成分。語彙の順序は p.9。

$h_2 = (0.1, 0.2, 0.3)^\top$ ：3 成分

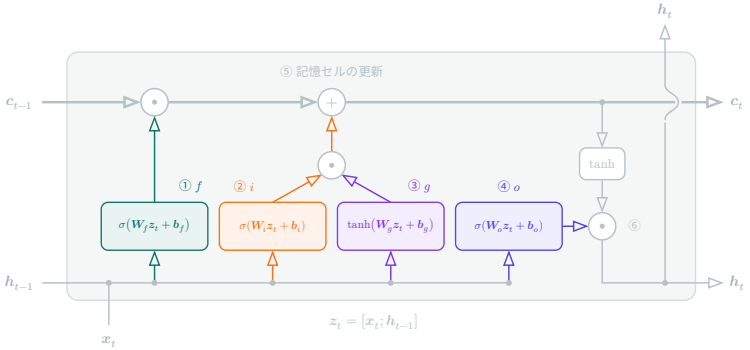
$z_3 = (0, 0, 0, 0, 0, 0, 1, 0, 0, 0.1, 0.2, 0.3)^\top$
隠れ状態の数値は説明用の仮の値。z_3 は 12 成分。

$$W_q z_t + b_q = W_{xq} x_t + W_{hq} h_{t-1} + b_q$$

$q \in \{f, i, g, o\}$ は計算する種類。$W_q = \begin{bmatrix} W_{xq} & W_{hq} \end{bmatrix}$ は $H \times (V + H)$。
連結により式を簡潔に書ける。各種類の重みとバイアスは、それぞれ学習する。

3 つのゲートと記憶セルの候補の計算

同じ連結入力 $z_t = [x_t; h_{t-1}]$ から、異なる重みで 4 つのベクトルを計算する。
sigmoid 関数 σ は各成分を 0〜1 の割合に、tanh は −1〜1 の候補値に変える。



① 忘却ゲート　記憶を保持する割合
$f_t = \sigma(W_f z_t + b_f)$

② 入力ゲート　候補を追加する割合
$i_t = \sigma(W_i z_t + b_i)$

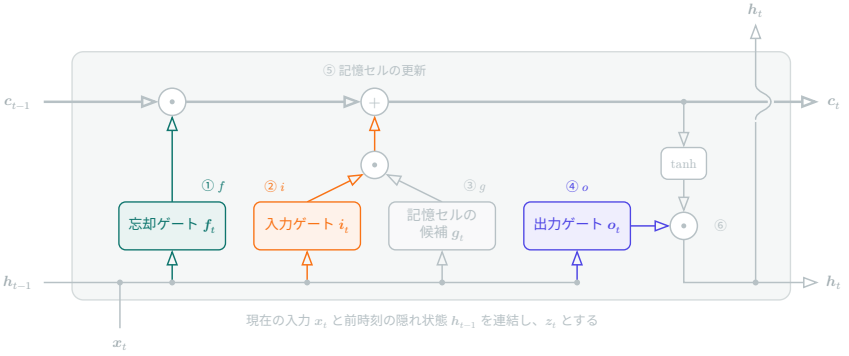
③ 記憶セルの候補　追加する情報
$g_t = \tanh(W_g z_t + b_g)$

④ 出力ゲート　隠れ状態に出す割合
$o_t = \sigma(W_o z_t + b_o)$

ゲートと候補の値は、入力と前時刻の隠れ状態に応じて変わる。種類ごとの重みとバイアスは、全時刻で共有して学習する。

3 つのゲートとアダマール積

ゲートは各成分を通す割合を 0〜1 の値で調整する。$\odot$ は同じ位置の成分同士を掛けるアダマール積。3 つのゲートがどの値に掛かるかを確認する。



p.33 と同じ構造。
色つきの 3 か所がゲート。

① f_t：忘却ゲート
$f_t \odot c_{t-1}$

② i_t：入力ゲート
$i_t \odot g_t$

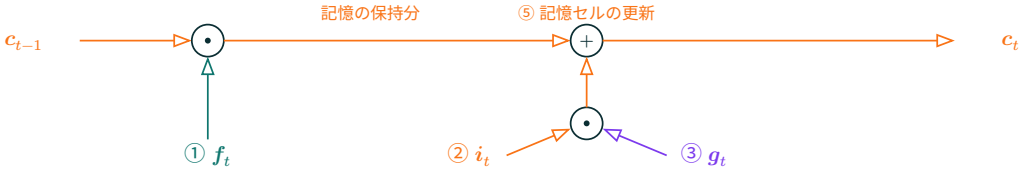
④ o_t：出力ゲート
$o_t \odot \tanh(c_t)$

$$\begin{pmatrix} f_{t,1} \\ \vdots \\ f_{t,H} \end{pmatrix} \odot \begin{pmatrix} c_{t-1,1} \\ \vdots \\ c_{t-1,H} \end{pmatrix} = \begin{pmatrix} f_{t,1}c_{t-1,1} \\ \vdots \\ f_{t,H}c_{t-1,H} \end{pmatrix}$$

成分を足し合わせず、元と同じ H 成分のベクトルを作る。
行列積とは区別する。

⑤ 記憶セルの更新

①〜③ で作った値と前時刻の記憶セルを使い、同じ位置の成分どうしで計算する。



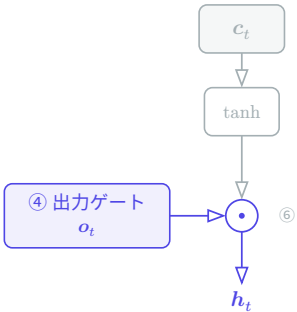
$$c_{t,j} = f_{t,j} c_{t-1,j} + i_{t,j} g_{t,j}$$

左の積は前時刻の記憶の保持分、右の積は候補からの追加分。成分 j は各ベクトルの同じ行の値だけで計算する。

記憶セルから隠れ状態を求める計算

記憶セルを tanh に通し、出力ゲートを成分ごとに掛ける。得られた隠れ状態を、予測と次時刻の計算に使う。

LSTM 内の計算経路



同じ成分同士の積

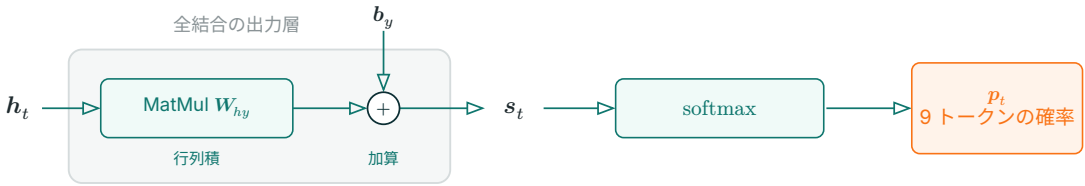
$$\underset{H\times 1}{\begin{bmatrix} h_{t,1} \\ \vdots \\ h_{t,j} \\ \vdots \\ h_{t,H} \end{bmatrix}} = \underset{H\times 1}{\begin{bmatrix} o_{t,1} \\ \vdots \\ o_{t,j} \\ \vdots \\ o_{t,H} \end{bmatrix}} \odot \underset{H\times 1}{\begin{bmatrix} \tanh(c_{t,1}) \\ \vdots \\ \tanh(c_{t,j}) \\ \vdots \\ \tanh(c_{t,H}) \end{bmatrix}}$$

$$h_{t,j} = o_{t,j} \tanh(c_{t,j})$$

成分数は H のまま。記憶セル c_t を保持しつつ、隠れ状態 h_t を出力層と次時刻へ渡す。

隠れ状態に基づく次のトークンの確率

LSTM で計算した隠れ状態 $\boldsymbol{h}_t$ を RNN と同じ出力層へ渡す。記憶セル $\boldsymbol{c}_t$ は出力層には直接入力しない。



$$\underbrace{\boldsymbol{s}_t}_{V\times 1} = \underbrace{\boldsymbol{W}_{hy}}_{V\times H}\,\underbrace{\boldsymbol{h}_t}_{H\times 1} + \underbrace{\boldsymbol{b}_y}_{V\times 1}$$

$$\boldsymbol{p}_t = \mathrm{softmax}(\boldsymbol{s}_t)$$

予測と学習の対応

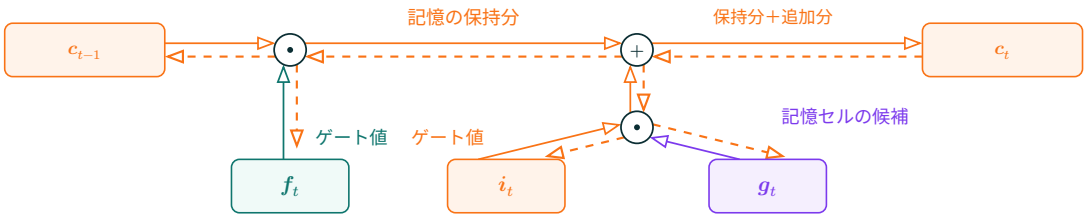
V は語彙数、H は状態の成分数。

$p_{t,k}$：次のトークンが index k である確率。
y_t：正解トークンの index。

各時刻の損失は $\ell_t = -\log p_{t,y_t}$。正解が $\langle \mathrm{pad} \rangle$ の時刻を除いた平均 L から、出力層と LSTM の重みへ逆伝播する。

LSTM の記憶セルを通る勾配の伝播

RNN で見た勾配消失を抑えるため、LSTM は記憶セルを次時刻へ伝える経路を持つ。
平均損失 L からの勾配は隠れ状態だけでなく記憶セルの経路も通って過去へ伝わる。



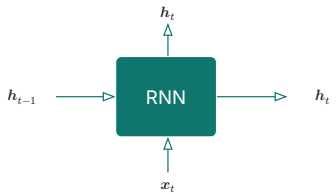
実線：順伝播 / オレンジの破線：逆伝播（勾配）。

この直接の経路では、各成分の勾配に忘却ゲートの値が掛かる。値が 1 に近いと、その成分の勾配を保ちやすい。

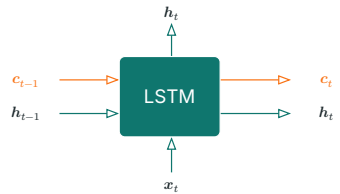
LSTM でも勾配消失が起きない保証はない。忘却ゲートの値や、ほかの計算経路にも依存する。

LSTM のまとめ：RNN との比較

RNN：隠れ状態を引き継ぐ



LSTM：記憶セルも引き継ぐ



項目	RNN	LSTM
受け渡す情報	隠れ状態 h_t	隠れ状態 h_t と記憶セル c_t
長期依存関係の学習	遠い時刻へ勾配が伝わりにくく、 離れた対応を学びにくい。	記憶を保つ直接経路により、 遠い時刻にも勾配を伝えやすくする。
生成するもの	次のトークンの確率分布。 選んだトークンを次の入力にする。	生成の手順は共通。 記憶セルも次の時刻へ引き継ぐ。

LSTM は残す記憶と出力する情報をゲートで調整する。次は学習済みモデルを使い、SMILES を生成する。

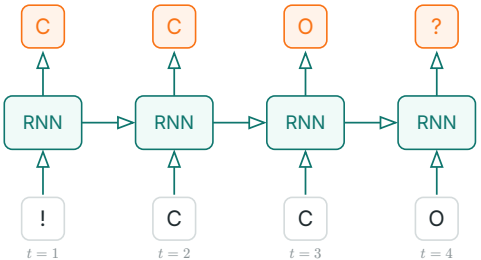
Part 4

SMILES の生成

学習済みモデルが出力する確率分布からトークンを選び、次の入力に用いる

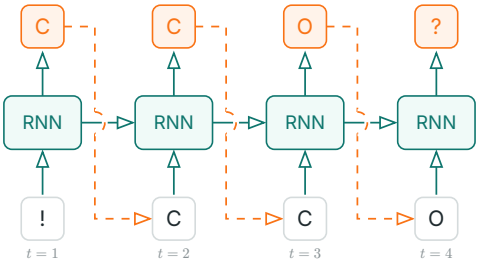
学習時と生成時の入力の違い

学習時の教師強制



入力はデータの正しいトークン。予測と正解を比較する。
系列をまとめて渡しても、内部の状態は時刻順に計算する。

生成時の自己回帰

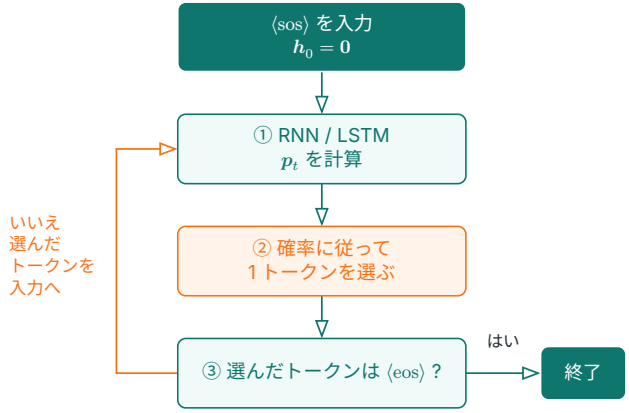


確率分布から選んだトークンを次の入力に戻す。これを自己回帰と呼ぶ。前時刻の選択が必要なので、1時刻ずつ順番に計算する。

学習では予測と正解のずれから重みを更新する。生成では学習済みの重みを固定し、選んだトークンと状態を次の時刻に渡す。

SMILES 生成の手順

学習した予測分布からトークンを選び、選んだトークンを次の入力に戻して生成する。



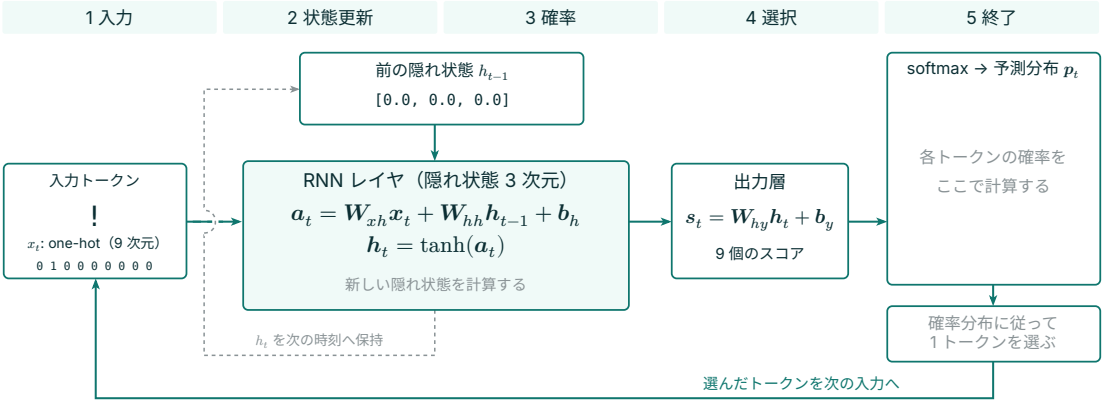
引き継ぐ情報と停止条件

- 終了記号までに選んだトークンを並べ、SMILES 候補にする。
- h_t を次時刻へ渡す。LSTM は c_t も渡す。
- この例では ⟨pad⟩ と ⟨sos⟩ も含む全語彙から選ぶ。
- 上限で打ち切った系列と、⟨eos⟩ で終わった系列を区別する。

特殊記号を生成候補から除く処理は任意。ここでは、処理を単純にするため、⟨sos⟩ や ⟨pad⟩ を生成対象から除外しない。

RNN によるエタノール 1 分子の生成

現在の文字列　!

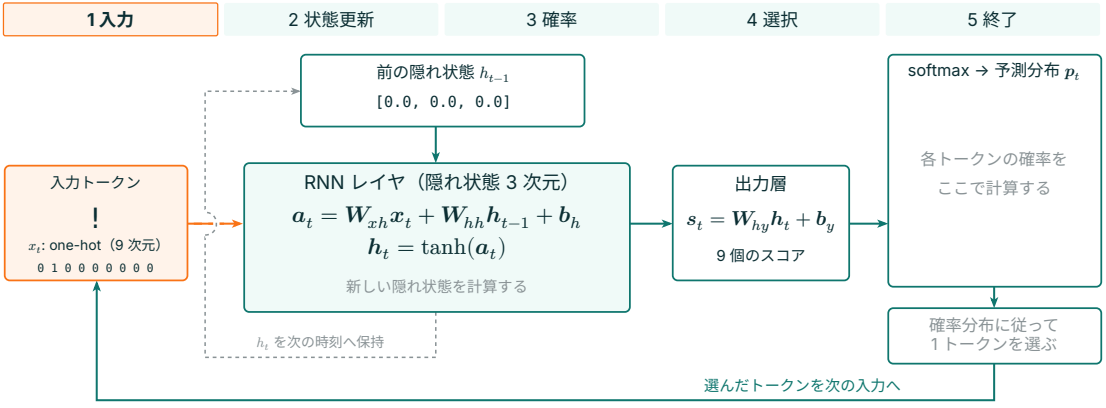


開始記号 "!" を入力し、C → C → O → 終了記号 "?" と選ぶ例を確認する。

隠れ状態は模式値、確率は説明用の例。全 9 種類から選び、開始・パディング記号も除外しない。

RNN の生成過程（時刻 1/4・入力）

現在の文字列　！



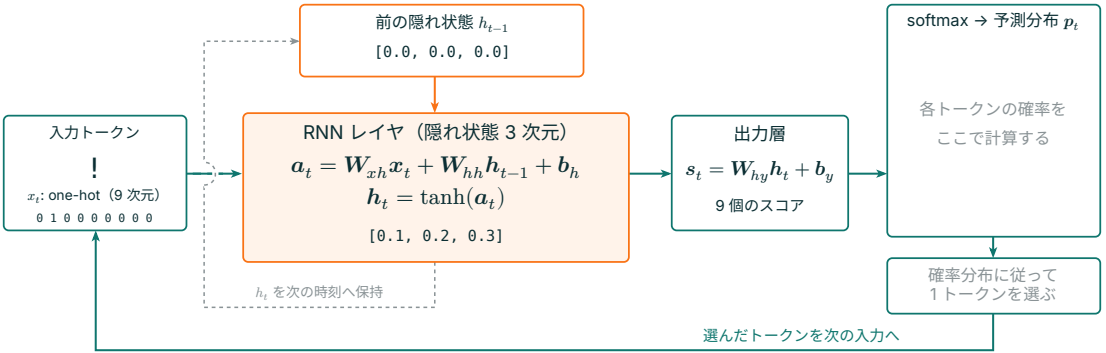
開始記号を one-hot ベクトルにして入力する。最初の隠れ状態は 0。

隠れ状態は模式値、確率は説明用の例。全 9 種類から選び、開始・パディング記号も除外しない。

RNN の生成過程（時刻 1/4・状態更新）

現在の文字列　！

1 入力 | **2 状態更新** | 3 確率 | 4 選択 | 5 終了



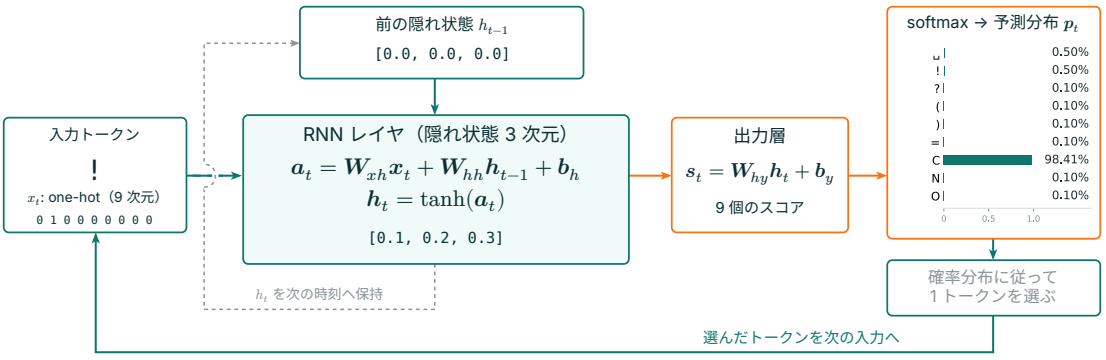
入力と前の隠れ状態から、新しい隠れ状態を計算する。

隠れ状態は模式値、確率は説明用の例。全 9 種類から選び、開始・パディング記号も除外しない。

RNN の生成過程（時刻 1/4・確率の計算）

現在の文字列　!

1 入力 | 2 状態更新 | **3 確率** | 4 選択 | 5 終了



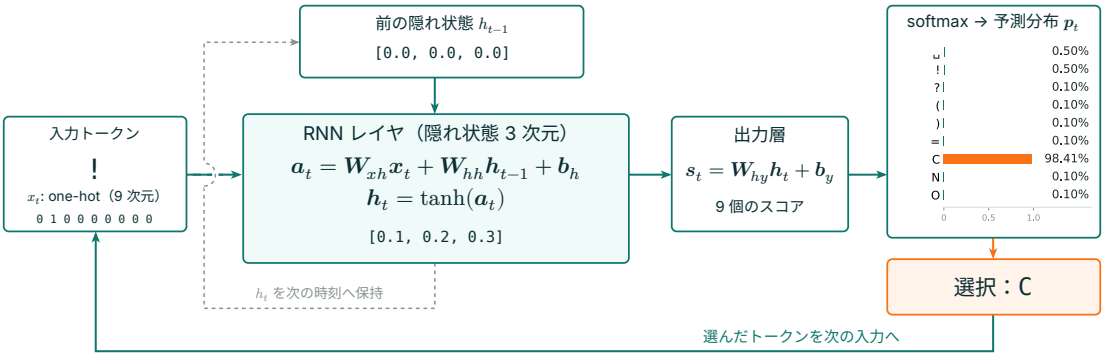
9 個のスコアを softmax で全 9 種類の確率に変換する。重みは更新しない。

隠れ状態は模式値、確率は説明用の例。全 9 種類から選び、開始・パディング記号も除外しない。

RNN の生成過程（時刻 1/4・トークンの選択）

現在の文字列　!C

1 入力 | 2 状態更新 | 3 確率 | **4 選択** | 5 終了

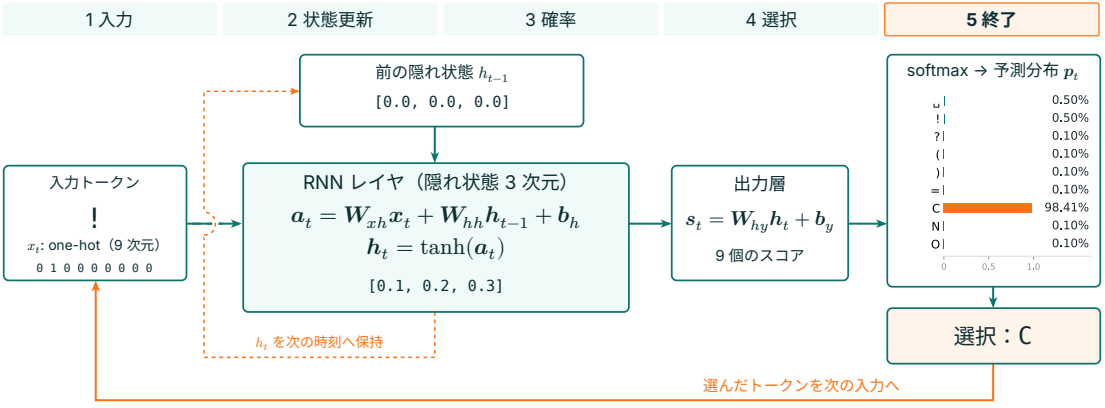


サンプリングの結果 "C" が選択された。

隠れ状態は模式値、確率は説明用の例。全 9 種類から選び、開始・パディング記号も除外しない。

RNN の生成過程（時刻 1/4・次時刻への引き継ぎ）

現在の文字列　!C

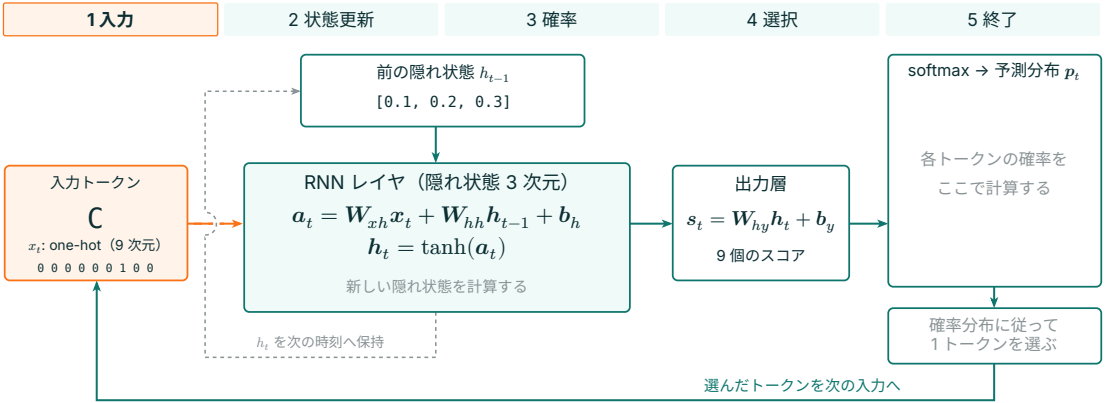


選んだトークンを次の入力へ戻し、隠れ状態も次の時刻へ引き継ぐ。

隠れ状態は模式値、確率は説明用の例。全 9 種類から選び、開始・パディング記号も除外しない。

RNN の生成過程（時刻 2/4・入力）

現在の文字列　!C

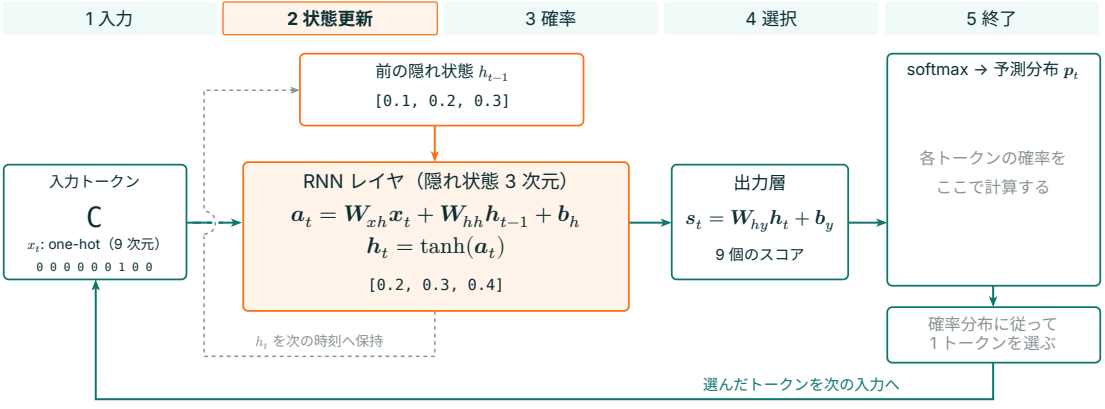


直前に選んだトークンを one-hot ベクトルに変換して入力する。

隠れ状態は模式値、確率は説明用の例。全 9 種類から選び、開始・パディング記号も除外しない。

RNN の生成過程（時刻 2/4・状態更新）

現在の文字列　!C



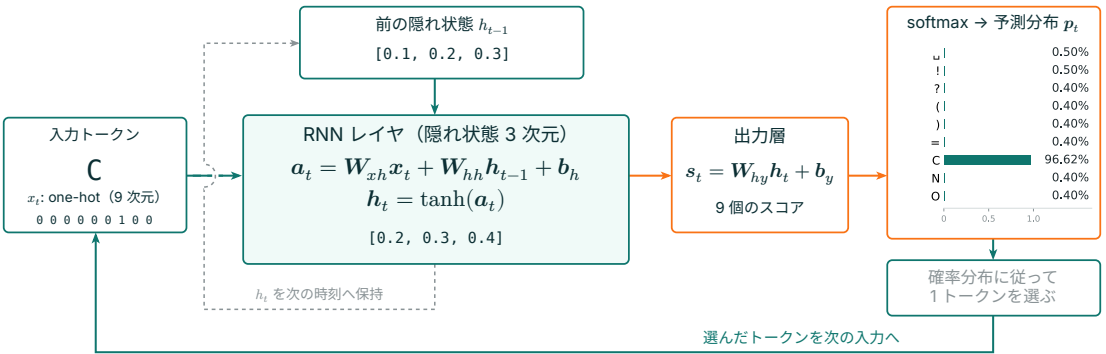
入力と前の隠れ状態から、新しい隠れ状態を計算する。

隠れ状態は模式値、確率は説明用の例。全 9 種類から選び、開始・パディング記号も除外しない。

RNN の生成過程（時刻 2/4・確率の計算）

現在の文字列　!C

1 入力 | 2 状態更新 | **3 確率** | 4 選択 | 5 終了



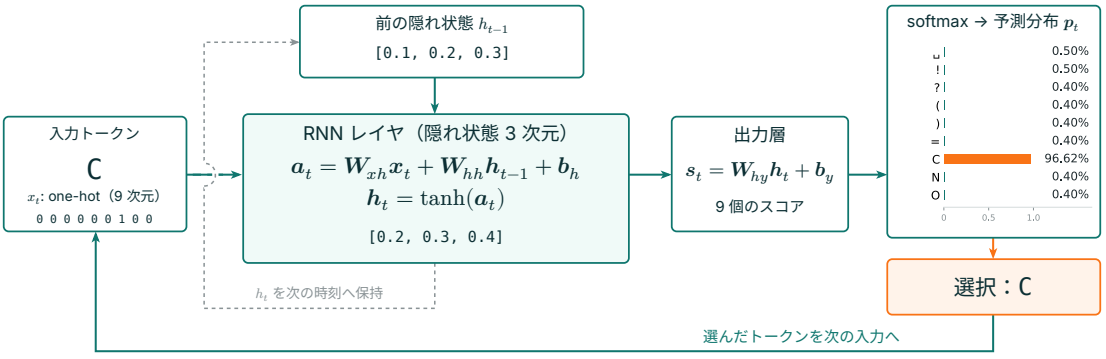
9 個のスコアを softmax で全 9 種類の確率に変換する。重みは更新しない。

隠れ状態は模式値、確率は説明用の例。全 9 種類から選び、開始・パディング記号も除外しない。

RNN の生成過程（時刻 2/4・トークンの選択）

現在の文字列　!CC

1 入力 | 2 状態更新 | 3 確率 | **4 選択** | 5 終了

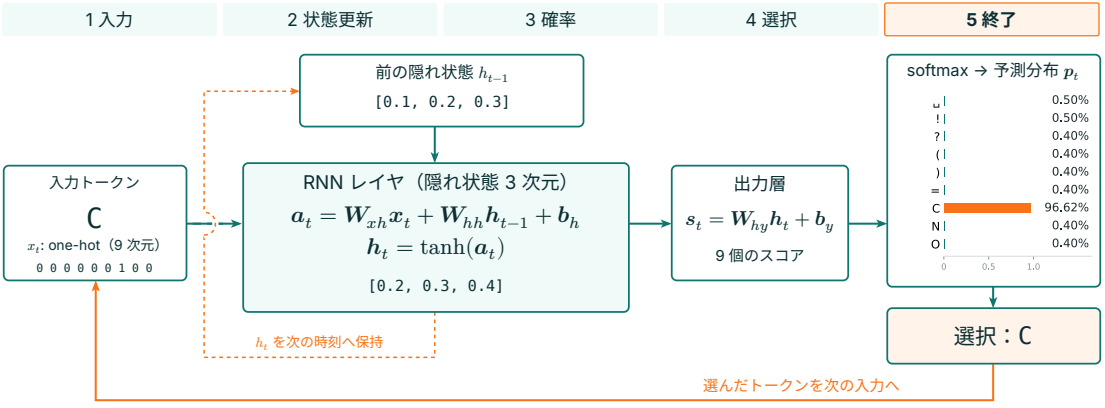


サンプリングの結果 "C" が選択された。

隠れ状態は模式値、確率は説明用の例。全 9 種類から選び、開始・パディング記号も除外しない。

RNN の生成過程（時刻 2/4・次時刻への引き継ぎ）

現在の文字列　!CC

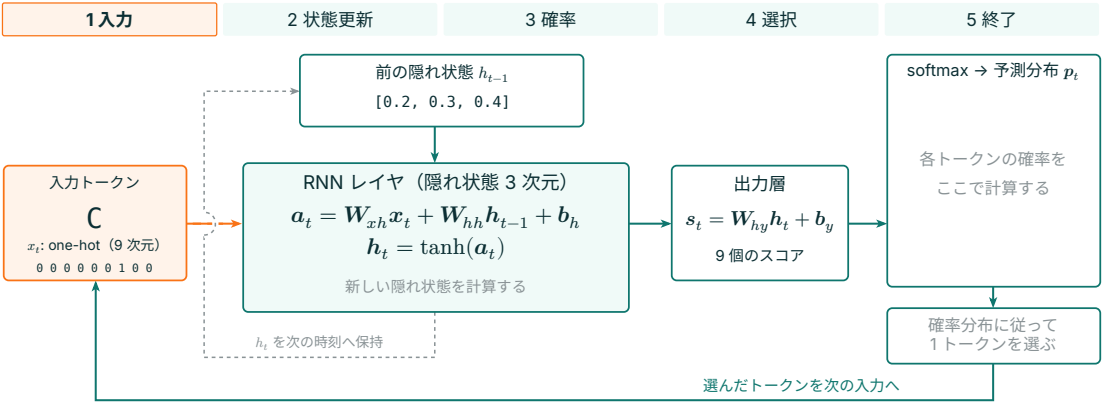


選んだトークンを次の入力へ戻し、隠れ状態も次の時刻へ引き継ぐ。

隠れ状態は模式値、確率は説明用の例。全 9 種類から選び、開始・パディング記号も除外しない。

RNN の生成過程（時刻 3/4・入力）

現在の文字列　!CC

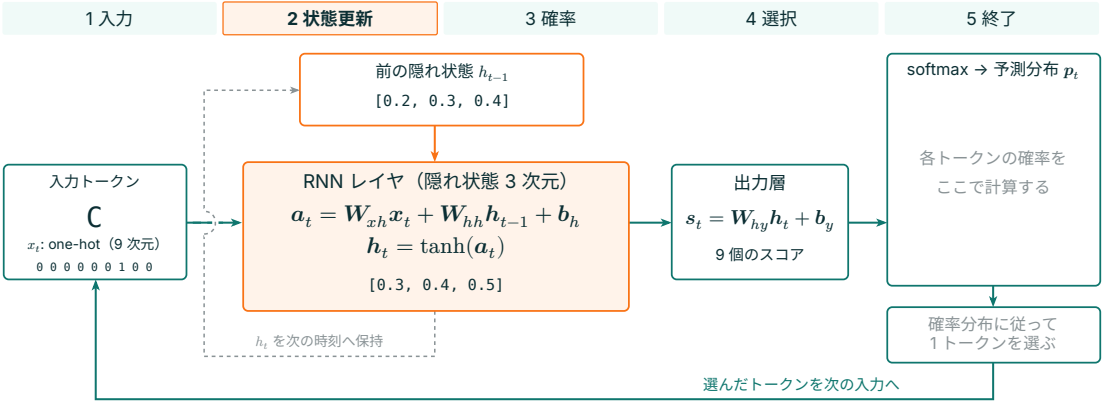


直前に選んだトークンを one-hot ベクトルに変換して入力する。

隠れ状態は模式値、確率は説明用の例。全 9 種類から選び、開始・パディング記号も除外しない。

RNN の生成過程（時刻 3/4・状態更新）

現在の文字列　!CC



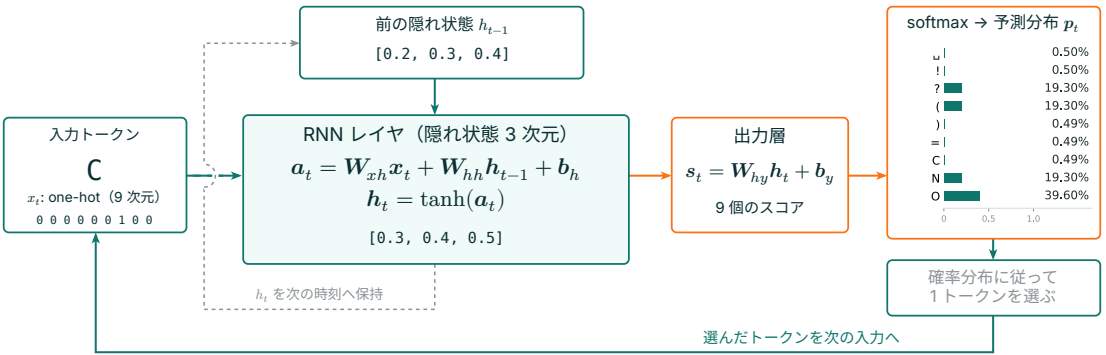
同じ "C" を入力しても、前時刻の隠れ状態が異なるため、更新後の隠れ状態も異なる。

隠れ状態は模式値、確率は説明用の例。全 9 種類から選び、開始・パディング記号も除外しない。

RNN の生成過程（時刻 3/4・確率の計算）

現在の文字列　!CC

1 入力 | 2 状態更新 | **3 確率** | 4 選択 | 5 終了



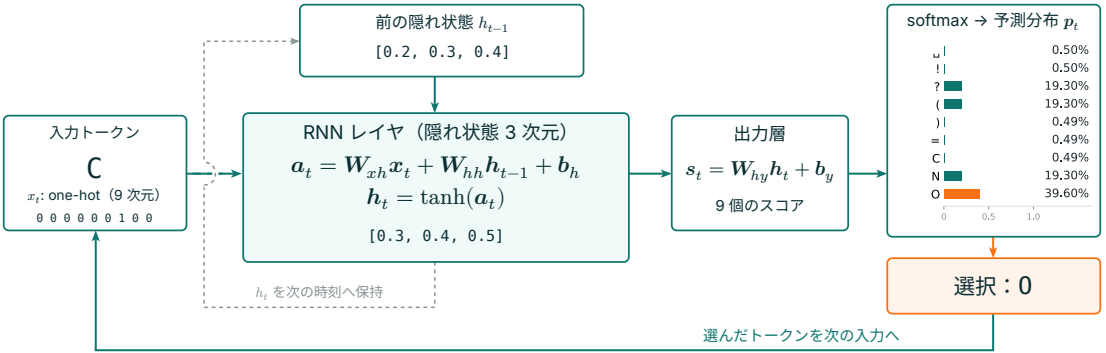
9 個のスコアを softmax で全 9 種類の確率に変換する。重みは更新しない。

隠れ状態は模式値、確率は説明用の例。全 9 種類から選び、開始・パディング記号も除外しない。

RNN の生成過程（時刻 3/4・トークンの選択）

現在の文字列　!CCO

1 入力 | 2 状態更新 | 3 確率 | **4 選択** | 5 終了

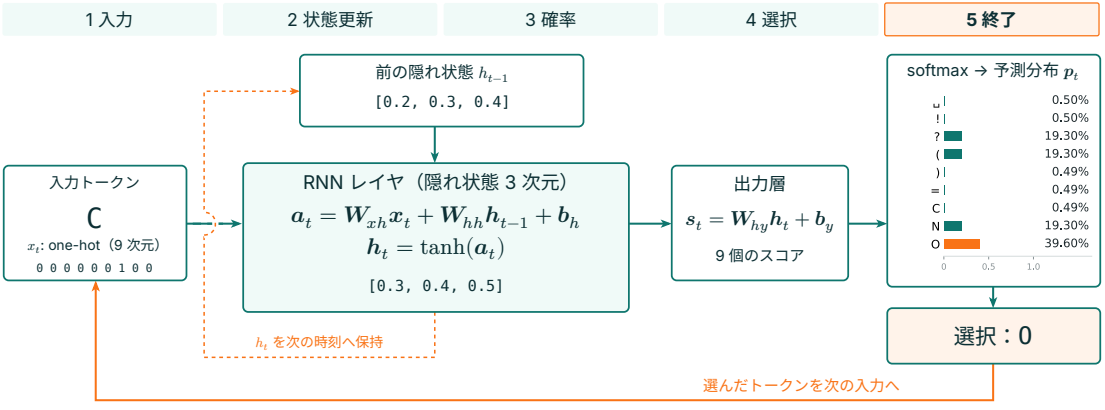


サンプリングの結果 "O" が選択された。

隠れ状態は模式値、確率は説明用の例。全 9 種類から選び、開始・パディング記号も除外しない。

RNN の生成過程（時刻 3/4・次時刻への引き継ぎ）

現在の文字列　!CCO

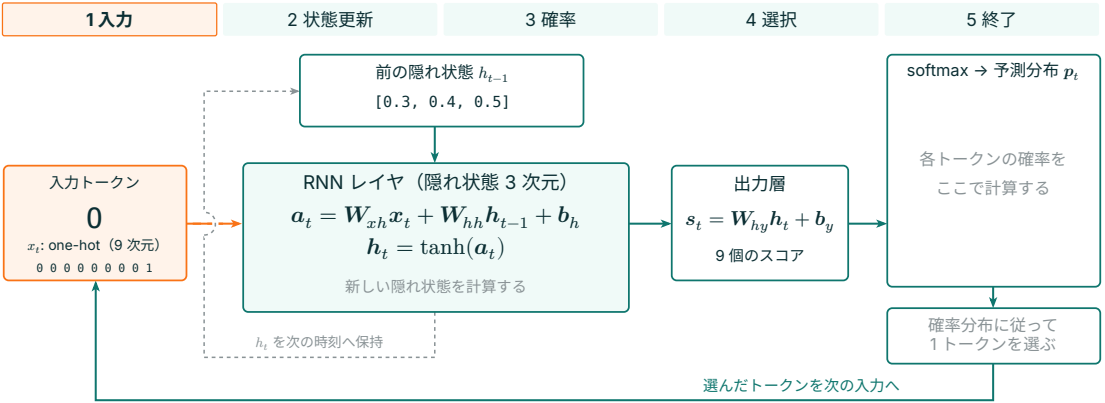


選んだトークンを次の入力へ戻し、隠れ状態も次の時刻へ引き継ぐ。

隠れ状態は模式値、確率は説明用の例。全 9 種類から選び、開始・パディング記号も除外しない。

RNN の生成過程（時刻 4/4・入力）

現在の文字列　!CCO



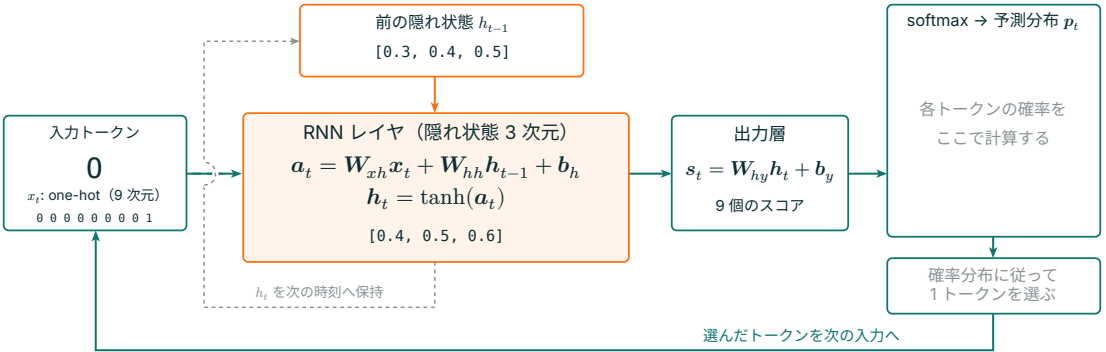
直前に選んだトークンを one-hot ベクトルに変換して入力する。

隠れ状態は模式値、確率は説明用の例。全 9 種類から選び、開始・パディング記号も除外しない。

RNN の生成過程（時刻 4/4・状態更新）

現在の文字列　!CCO

1 入力 | **2 状態更新** | 3 確率 | 4 選択 | 5 終了



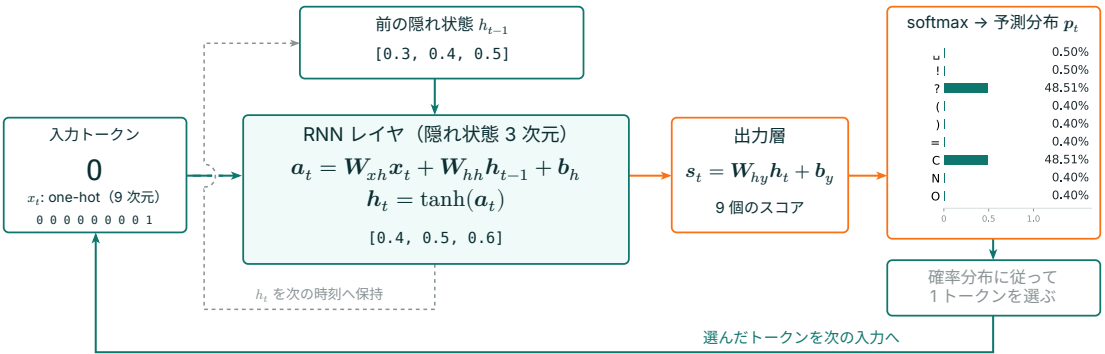
入力と前の隠れ状態から、新しい隠れ状態を計算する。

隠れ状態は模式値、確率は説明用の例。全 9 種類から選び、開始・パディング記号も除外しない。

RNN の生成過程（時刻 4/4・確率の計算）

現在の文字列　!CCO

1 入力 | 2 状態更新 | **3 確率** | 4 選択 | 5 終了



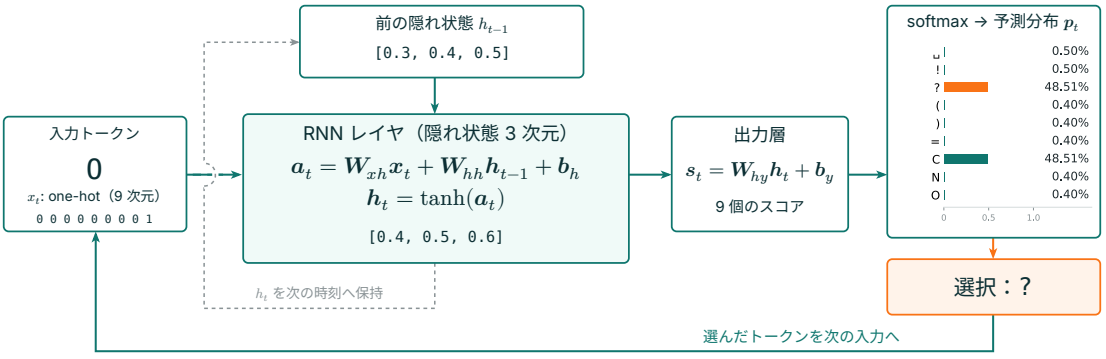
9 個のスコアを softmax で全 9 種類の確率に変換する。重みは更新しない。

隠れ状態は模式値、確率は説明用の例。全 9 種類から選び、開始・パディング記号も除外しない。

RNN の生成過程（時刻 4/4・トークンの選択）

現在の文字列　!CCO?

1 入力 | 2 状態更新 | 3 確率 | **4 選択** | 5 終了



サンプリングの結果 "?" が選択された。

隠れ状態は模式値、確率は説明用の例。全 9 種類から選び、開始・パディング記号も除外しない。

RNN の生成過程（時刻 4/4・終了判定）

現在の文字列　!CCO?

1 入力 | 2 状態更新 | 3 確率 | 4 選択 | **5 終了**

前の隠れ状態 h_{t-1}

[0.3, 0.4, 0.5]

入力トークン

0

x_t: one-hot（9 次元）

0 0 0 0 0 0 0 0 1

RNN レイヤ（隠れ状態 3 次元）

$$a_t = W_{xh}x_t + W_{hh}h_{t-1} + b_h$$

$$h_t = \tanh(a_t)$$

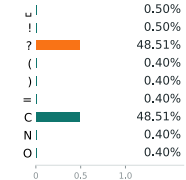
[0.4, 0.5, 0.6]

出力層

$$s_t = W_{hy}h_t + b_y$$

9 個のスコア

softmax → 予測分布 p_t



選択：?

生成終了　CCO　エタノール

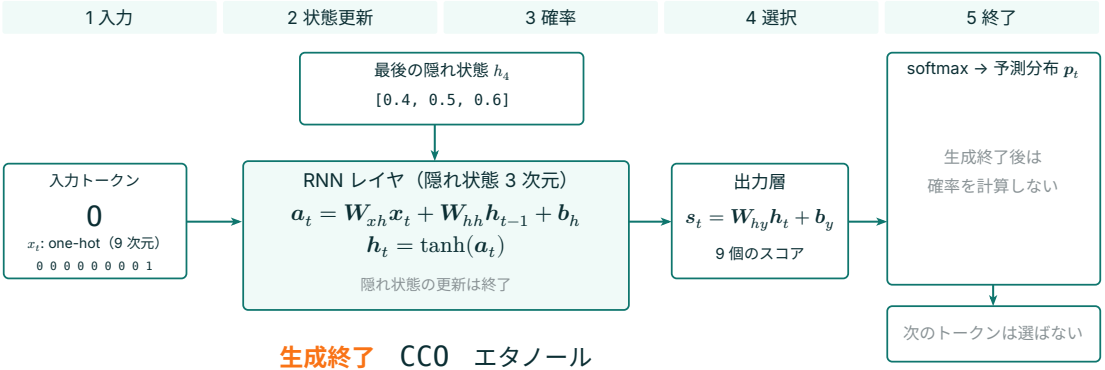
開始・終了記号は SMILES に含めない。

終了記号を選んだら停止する。次の RNN 計算へは進まない。

隠れ状態は模式値、確率は説明用の例。全 9 種類から選び、開始・パディング記号も除外しない。

終了記号による生成の停止

現在の文字列　!CCO?



生成終了　CCO　エタノール

開始・終了記号は SMILES に含めない。

終了記号を選んだら停止する。次の RNN 計算へは進まない。

隠れ状態は模式値、確率は説明用の例。全 9 種類から選び、開始・パディング記号も除外しない。

分子構造生成モデルのまとめ

SMILES をトークン列に変換し、次のトークンの予測を学習する。その予測を繰り返すことで、分子を表す文字列を生成できる。

学んだこと	要点
入力の準備	トークン化 → index → one-hot ベクトル。特殊記号で開始・終了・長さを扱う。
モデルと学習	RNN は隠れ状態、LSTM は記憶セルとゲートも使う。平均損失の勾配から重みを更新する。
分子の生成	学習済みの重みを固定し、予測分布からの選択と状態の更新を $\langle \mathrm{eos} \rangle$ まで繰り返す。

学習の目標

学習データの系列に高い確率を与える。各時刻の交差エントロピーを平均した L を小さくすることに対応する。

生成した分子の確認

生成した SMILES を解釈し、分子の妥当性、重複、学習データに対する新規性を確かめる。

参考資料

ゼロから作る Deep Learning

斎藤 康毅 著、オライリー・ジャパン、2016 年。参照：3.5 節（softmax）、4.2.2 節（交差エントロピー）。

ゼロから作る Deep Learning ❷ —自然言語処理編

斎藤 康毅 著、オライリー・ジャパン、2018 年。参照：第 5 章、第 6 章、第 7 章。

機械学習による分子最適化 —数理と実装—

梶野 洸 著、オーム社、2023 年。参照：第 4 章。